

DETECTING INTRUDERS IN YOUR SERVER



October 16, 2025

“The one where we catch them red-handed”

Credits

Content: Sebastian Garcia, Veronica Valeros, Maria Rigaki
Martin Řepa, Lukáš Forst, Ondřej Lukáš, Muris Sladić

Illustrations: Fermin Valeros

Design: Veronica Garcia, Veronica Valeros, Ondřej Lukáš

Music: Sebastian Garcia, Veronica Valeros, Ondřej Lukáš

CTU Video Recording: Jan Sláma, Václav Svoboda, Marcela Charvatová

Audio files, 3D prints, and Stickers: Veronica Valeros

CTU Video Recording: Jan Sláma, Václav Svoboda, Marcela Charvatová

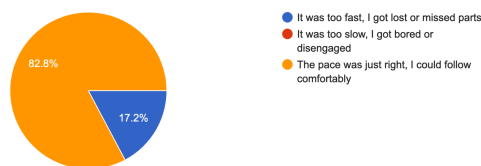
Audio files, 3D prints, and Stickers: Veronica Valeros

LESSON 4 / DETECTING INTRUDERS IN YOUR SERVER

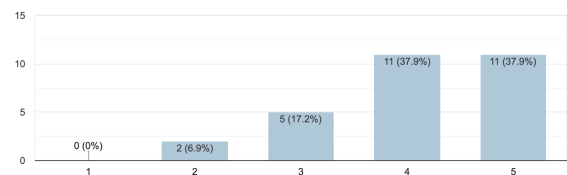
CLASS DOCUMENT	https://bit.ly/BSY2025-4
WEBSITE	https://cybersecurity.bsy.fel.cvut.cz/
MATRIX	https://matrix.bsy.fel.cvut.cz/
CTFD (CTU STUDENTS)	https://ctfd.bsy.fel.cvut.cz/
PASSCODE FORM (MOOC STUDENTS)	https://bit.ly/BSY-MOOCPasscode
FEEDBACK	https://bit.ly/BSY-Feedback
LIVESTREAM	https://bit.ly/BSY-Livestream
INTRO Animation	https://bit.ly/BSY-IntroVideo
VIDEO RECORDINGS PLAYLIST	https://bit.ly/BSY-Recordings

Results from last class survey (14:32, 1m)

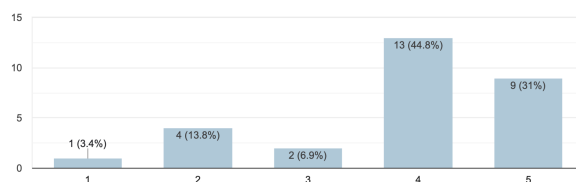
How was the pace of the class?
29 responses



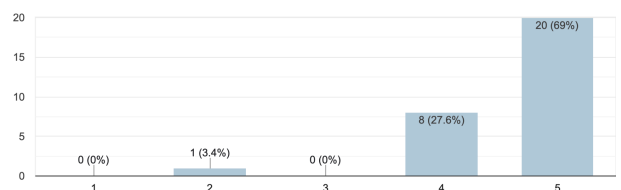
Were you able to follow what the teacher explained during class?
29 responses



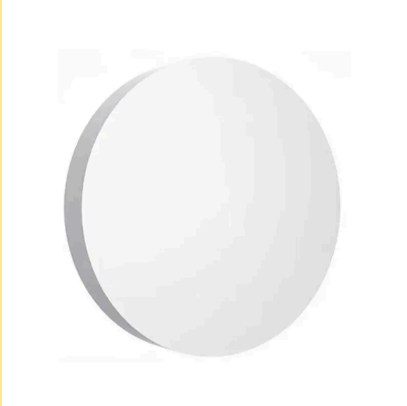
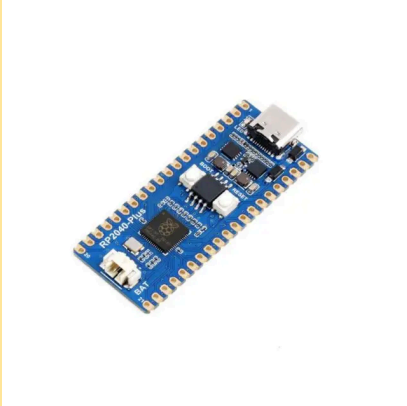
Were you able to follow along and run the commands during class on your device?
29 responses



How useful was the class document and other supporting resources shared by the teachers?
29 responses



Pioneer Prize for Assignment 3 (14:33, 2m)

1 st Place	2 nd Place	3 rd Place
		
ESP32 Development Board	Zigbee Smart Button	Waveshare Pico-like MCU board 16 MB
kupkajan	hollmmax	zahorto2

Main topics for this class

1. [How to find out if someone got into your computer?](#)
2. [How to secure your computer?](#)
3. [Host-Based Intrusion Detection Systems: AIDE](#)

Parish Notices (14:35, 1m)

- MOOC Students:
 - Please start Class 04 in SCL now.
 - If your internet is slow, you can [install AIDE](#). [Lynis](#). [Logwatch](#)

- CTU Students: Remember to update bsy-clippy in your CTU Docker
 - `pip install --upgrade bsy-clippy --break-system-packages`

Protection is about layers (14:36, 3m)

Goal: To understand how to protect a computer. To know how to find if there was an intruder inside a computer.

Security should be a layered approach:

1. Assume that all other protection measures have **failed**, and each one is the **last** one.
2. Separate your defenses so they don't **depend** on each other.
3. Protect every aspect you can think of. And then again, some you didn't think of.
4. Typically separated into approximately these steps:
 - a. **Data** protection (encryption, hiding, backup, etc.)
 - b. **Application** protection (updates, monitoring, logs, processes, kernel, authorization, etc.)
 - c. **Service** protection (DB, load balancers, etc.)
 - d. **Endpoint** protection (FW, users, authorizations, monitoring, logs, AV, EDR, etc.)
 - e. **Connection** protection (WiFi, VLANs, etc.)
 - f. **Humans**
5. The famous concept of **Zero-trust** refers to this. No part of the system should trust any other part of itself. Neither internal nor external.
 - a. Of course, impossible.



How to find out if someone got into your computer? (14:39, 1m)

Every access to a computer leaves **traces** in someplace. Attackers can gain access using standard authentication methods, exploits, backdoors, or by stealing your hard disk. You should be aware of all of them.

Start by looking for signs of intrusions, backdoors, or anything out of place.



Check logins with standard methods (14:40, 10m)

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

1. Standard methods are those controlled by the operating system: **SSH, Telnet, SMB, NFS, etc.**
2. Non-standard methods include backdoors, exploits, and other similar techniques. Those are **not** recorded by the system logs.
3. Check who is logged in now:
 - a. `w`
 - i. In Docker, there are some limitations. CTU Dockers may show this empty.
 - b.  **AI View. Let's see what clippy has to say**
 - i. `w | bsy-clippy -c -u 'You are a cybersecurity expert. Tell me if there is any indication of compromise. Check the login times if they make sense' -r 10`
 - ii. **CTU Students: Docker container. ▾**: use the cesnet profile
 1. `w | bsy-clippy -c -u 'You are a cybersecurity expert. Tell me if there is any indication of compromise. Check the login times if they make sense' -r 10 --profile cesnet`
4. Check the last logins from all computers:
 - a. `last`
 - i. List past logins on the system one after the other.
 - b. This command reads log files that store login system information:

- i. `/var/log/utmp`¹: Binary file. A full accounting of the current status of the system, system boot time (used by uptime), user logins, logouts, etc. **It may not exist.**
 - ii. `/var/log/wtmp`: Binary file. Acts as a historical file to record all login sessions, including remote logins (via SSH), local logins, reboots, and shutdowns.
 - iii. To remember: **u**tmp (users), **w**tmp (who), **b**tmp (bad).
- c. Logrotate
- i. If logs are rotated by logrotate, then `last` shows the last one.
 - ii. To see an old log, specify the log file to read from with `-f`
 - 1. `last -f /var/log/utmp.1`
 - a. May not exist.
 - iii. If they are older, logrotate can compress them. Check:
 - 1. `vi /etc/logrotate.conf`
 - 2. `ls /etc/logrotate.d/`
 - 3. `vi /etc/logrotate.d/btmp`
 - iv. If you have compressed logs, to see them, first uncompress them and then read them:
 - 1. `gunzip -k /var/log/btmp.2.gz`
 - 2. `zcat /var/log/btmp.2.gz`
5. Check failed attempts to log in
- a. `lastb`
 - i. Uses `/var/log/btmp`: records failed login attempts
6. Another way is to see the list of users and then when each of them logged in for the last time:
- a. `lastlog`



What is the disadvantage of using the commands `w`, `last`, and `lastlog`?

¹ B, M. (2016). Torvalds Tuesday: Logon History in the *tmp Files - Matt B - Medium. [online] Medium. Available at: <https://bromiley.medium.com/torvalds-tuesday-logon-history-in-the-tmp-files-83530b2acc28> [Accessed 16 Oct. 2024].

Check the existing users in the system and password database (root needed) (14:50, 5m)

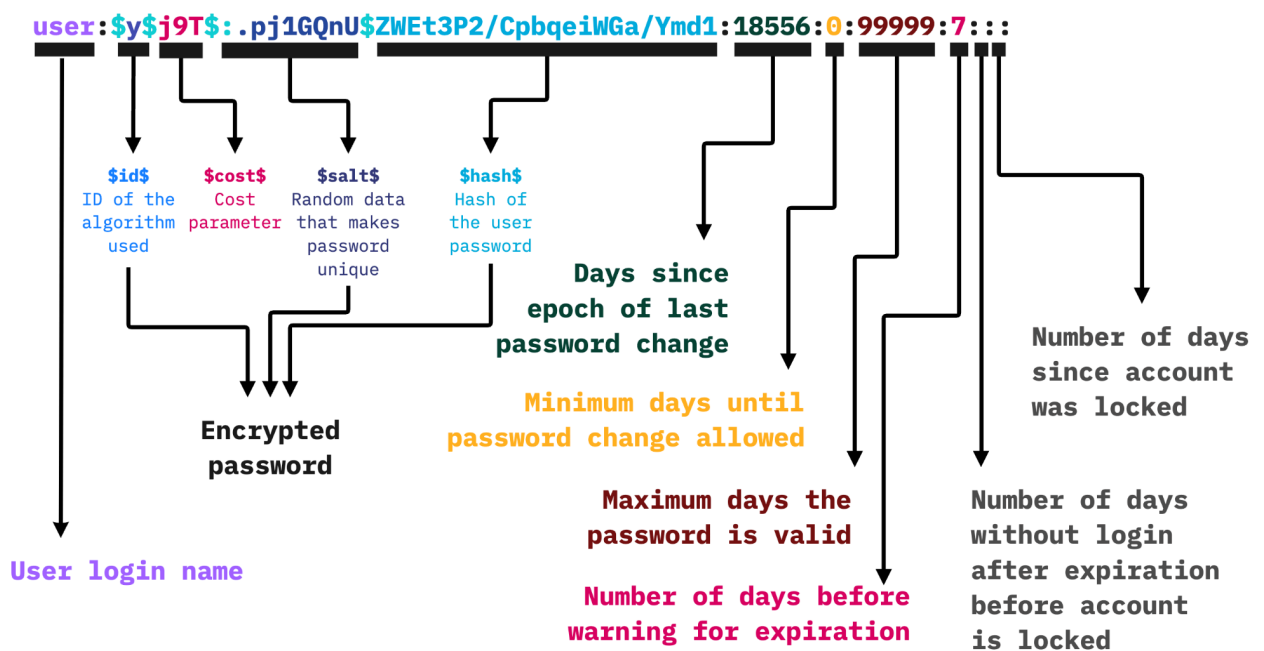
CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

Users and passwords in the system are kept in the **/etc/shadow** file

Let's inspect it

- `cat /etc/shadow`



1. The first field is the **user name**.
2. The second field is important. It is a composed value `<ID>.<COST><SALT>.<HASH>`
 - a. If the whole field is `*`, it means the user does not have a password assigned.
 - b. The **ID** field can have different values:
 - i. `1` = MD5
 - ii. `5` = SHA-256
 - iii. `6` = SHA-512

LESSON 4 / DETECTING INTRUDERS IN YOUR SERVER

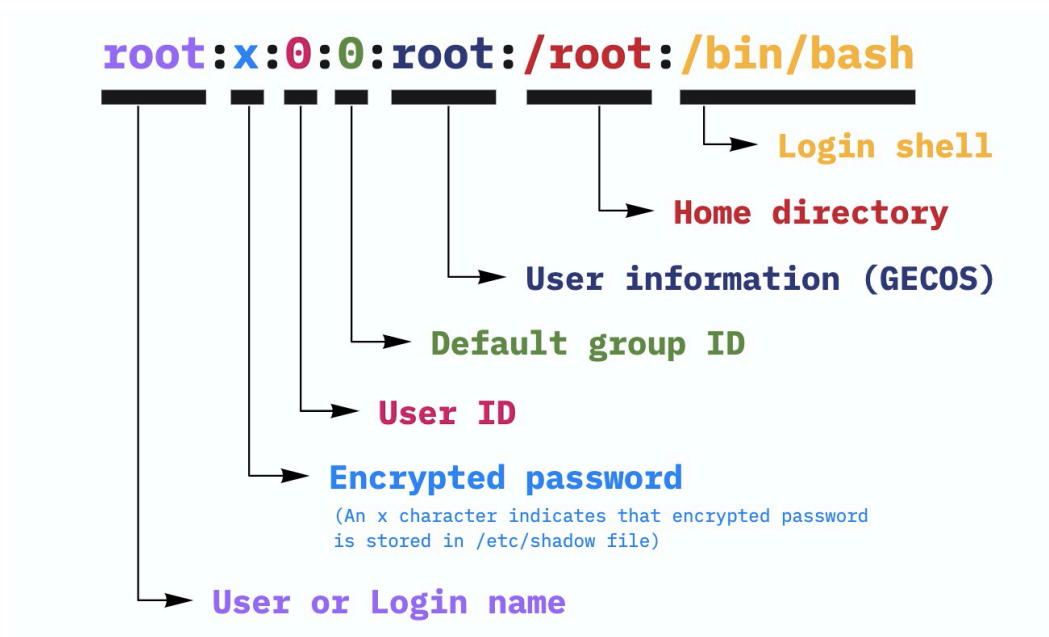
- iv. `$2a$, $2b$, $2y$`: bcrypt (originally based on 'Blowfish', corrected, alternate)
- v. `y` = yescrypt
 - 1. Evolution of `scrypt`. Cost affects memory and CPU. Adjustable cost. GPU/FPGA resistant.
- vi. `7` = scrypt
- c. The **COST** field
 - i. How computationally expensive is it to compute the hash? (In `Yescrypt`, it affects both CPU and Memory). Without storing the cost, the system wouldn't know how many rounds or how much computational effort to apply when re-hashing the user's input.
- d. The **Salt** field
 - i. Adds a pseudo-random value that prevents the use of precomputed hashes (rainbow tables).
- 3. To quickly disable a user so it can no longer log in without forgetting the password, put a **! (exclamation mark)** as the first character of the password field:
 - a. `user: !$6$<string>:18071:0:99999:7:::`
- 4. Any invalid format means the user cannot log in.

Check the users in the system, but only their details (no need to be root) (14:55, 2m)

Some user information is stored in the `/etc/passwd` file. It is a file that contains information about **users**, but **not** their passwords.

Let's inspect the content of the shadow file

- `cat /etc/passwd`



We will see later that file permissions are in effect, but the `/etc/passwd` file has permission to be read by any user. Why?

Find and kill processes (14:57, 5m)

Attackers may execute code on the computer, and that code can be a process running in memory. Even if the binary file was deleted.

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

1. Search for their processes:

- a. `ps afxu`

- i. `-a`: all users
- ii. `-f`: Display the uid, pid, parent pid, recent CPU usage, process start time, controlling tty, elapsed CPU usage, and the associated command
- iii. `-x`: include processes that do not have a controlling terminal

- b. `htop`

LESSON 4 / DETECTING INTRUDERS IN YOUR SERVER

- i. Htop shows processes too, but it uses only `/proc/[pid]/cmdline`, while ps also uses `/proc/[pid]/stat`. So they may show different process names.
- c. `btop` (already installed in the Dockers)
 - i. Probably the most modern resource monitor in Linux.
 - ii. `apt install btop`
- 2. Kill processes you know are suspicious
 - a. `kill -9 <process id>`
 - i. `-9` is a signal sent to the process. `-9` is special because it means ‘die,’ but it can not be intercepted by the process and stopped.
 - ii. `-1` is ‘die,’ but the process can intercept it and choose to ignore it.
 - b. To kill all the processes of a user:
 - i. `killall -u <username>`
 - c. Let’s try
 - i. Execute something forever as a new background process.
 - 1. `nohup bash -c "while true; do echo '1' >> /tmp/file; sleep 5; done" &`
 - a. **nohup**: Invoke a utility immune to hangups by redirecting its standard output (terminal) to the file called “nohup”.
 - b. **bash -c**: Executes this as a command in bash.
 - c. **>>**: Redirects the standard output of the command (echo in this case) to a file (/tmp/file) but it appends to previous content (Only one **>** is to rewrite).
 - d. **&**: Detach this command from the shell and puts it running in the background.
 - ii. Check it works
 - 1. `tail -f /tmp/file`
 - a. This outputs the last line (by default, 1 line) of the file (tail) and waits for more lines to arrive, so it can output them. This *follows* the new content of the file.

- iii. Task for you: find the process responsible and kill it
- iv. How to solve
 - 1. `ps afx`
 - 2. Identify the correct process, the one with 'while' in it. Not your main bash.
- v. Now kill it
 - 1. `kill -9 <process id (pid)>`
 - a. -9: Sends the signal KILL (that is not ignorable)
 - b. By default, kill sends -1 (HUP), but the process can ignore it.



Check log files (15:02, 4m)

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

All the logs of Linux systems are located in `/var/log/`. Let's see some of the logs that contain login information:

1. Authentication information is in `/var/log/auth*`
 - a. `grep user /var/log/auth.log | grep -vi cron | less`
 - i. `grep -v cron`: filter OUT the lines containing 'cron'
 - b. If there are rotated files:
 - i. `grep user /var/log/auth.log.1 | grep -vi cron | less`
 - c. If compressed
 - i. `zgrep user /var/log/auth.log.2.gz | grep -vi cron | less`
2. Find failed attempts to log in:
 - a. `grep -i "Failed password" /var/log/auth.log`
3. The logs for all the systems are in `/var/log/syslog*`
 - a. `cat /var/log/syslog | less`
4. Remember, you can always use Clippy and ask for help.
 - a. To better see the `/var/log/syslog`, you can use (not in our dockers)

- i. `journalctl --since "24 hours ago"`
5. You can also try the `dmesg` command in normal Linux to print the kernel ring buffer (kernel logs), but **not** in Docker, since this interface is not accessible!
6. 🤖 AI View:
 - a. `cat /var/log/auth.log |bsy-clippy -c -u 'You are a cyber security expert. Tell me if there is any indication of compromise. Check the login times if they make sense' -r 10`
 - b. CTU Students: Docker container. ▾ `cat /var/log/auth.log |bsy-clippy -c -u 'You are a cyber security expert. Tell me if there is any indication of compromise. Check the login times if they make sense' -r 10 --profile cesnet`



Find modified files in the system (15:06–33, 4m)

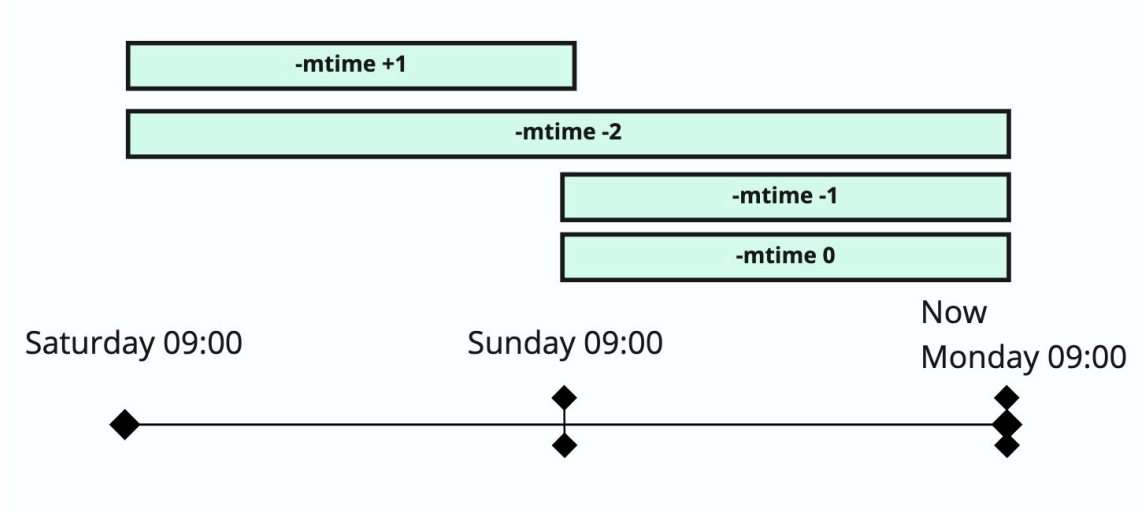
CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

It is critical to know what has changed after an attack or infection.

1. `find`²: a powerful tool for searching for files and much more!
 - a. The parameter is `-mtime [+|-]N`
 - i. If the file was modified **more (+)** or **less (-)** than **N** days ago.
 - ii. When ‘find’ figures out how many *24-hour periods* ago the file was last accessed, any fractional part is ignored.
 - iii. So, for a file to match `-mtime +1` (in your mind, “files modified more than 1 day ago”), a file has to have been accessed at **least 1 day** ago (because if not, its mtime can be, for example, 1.5days).
 - iv. Without `+|-` it means **exactly** $N \times 24$ hs ago. 0 means in the **last** 24hs
 - v. Be careful. It is not particularly precise, and we are unsure *why*. 😞
 1. (wait, did you just italicized emojis? Yes, I did.
 2. (wait, can you say italicized? For sure, I can)

² Cheatsheet for find linux (2024) [online] Gist. Available at: <https://gist.github.com/gr1ev0us/3a9b9d9dbdd38f6379288eb2686fc538> [Accessed 17 Oct. 2024].

2. Understanding how `find` works with the `-mtime` parameter:3. Examples of how to use `find` to check for modified files:

a. Files that were modified 24 hours ago:

i. `find / -mtime 0 -ls | grep -v "proc\|sys" | less`

b. Files modified less than 48 hours ago:

i. `find / -mtime -2 -ls | grep -v "proc\|sys" | less`

c. Files modified in the last week (less than 7 days ago):

i. `find / -mtime -7 -ls | grep -v "proc\|sys" | less`

4. An alternative for individual files to know when it was last modified

a. `stat /etc/passwd`

```
File: /etc/passwd
Size: 1033      Blocks: 8      IO Block: 4096   regular file
Device: 0,77   Inode: 932230   Links: 1
Access: (0644/-rw-r--r--)  Uid: (  0/   root)   Gid: (  0/   root)
Access: 2025-10-05 14:19:19.000000000 +0000
Modify: 2025-10-05 14:19:19.000000000 +0000
Change: 2025-10-05 14:19:43.962106010 +0000
Birth: 2025-10-05 14:19:43.962106010 +0000
```

- **Modify:** When the **content** was last changed.
- **Change:** When the **metadata** was last changed (permissions, ownership, etc.)



Miscellaneous checks (15:10, 3m)

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

1. Past commands.

LESSON 4 / DETECTING INTRUDERS IN YOUR SERVER

- a. `less ~/.bash_history`
 - i. `~` means 'the home folder of the user'.
 - ii. If the first letter of the filename is `'.'` then the file does not show in the traditional `'ls'`, unless `'-a'` is used. AKA 'hidden'.
 - iii. In SCL, you don't have stored commands until the terminal ends. You can try to inject some with
 1. `bash`
 2. `ps afx`
 3. `w`
 4. `CTRL-D`
 5. `less ~/.bash_history`
2. Installed programs with the date of installation. Using Ubuntu/Debian style
 - a. `zgrep " install " /var/log/dpkg.log`
3. Open ports
 - a. If an attacker gets in, a process may be listening for connections.
 - i. `ss -anpult`
 1. Replaces old `netstat -anp`
 2. `-a`: All. Display both listening and not listening
 3. `-n`: Do not resolve DNS
 4. `-p`: Show processes using a socket.
 5. `-l`: Show processes listening in a port.
 6. `-t`: tcp
 7. `-u`: udp
 - b. What is important? Search for
 - i. LISTEN processes in TCP
 - ii. Any UDP process
 - c. How to find which process opened a port
 - i. `ss -p` or `netstat -p`
 - ii. If not `lsof -i :22`
 - iii. (Docker daemon uses the IP 127.0.0.11 and port 60879/UDP as an internal resolver, which is why it doesn't have a process)
4. Firewall

- a. A firewall filters packets in and out of a system.
- b. We may offer an optional class on this.
- c. Check this for now: <https://www.tecmint.com/linux-iptables-commands/>



Common backdoors? (15:13, 6m)

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

The idea is that attackers may have left behind methods to gain access to the system later. These are referred to as ‘backdoors’.

This is not easy because there are many options.

- Inserted SSHd authentication keys
 - `cat ~/.ssh/authorized_keys`
 - These are the public keys authorized to log in without a password.
 - **!!!CTU Students: DO NOT remove the teachers' SSH keys.!!!**
- Processes executed at login. There are many places to check, some are:
 - `less /etc/rc.local` (may not exist)
 - `less /etc/profile`
 - `ls /etc/profile.d/*`
 - `cat ~/.bashrc`
 - `~`: home of the user.
 - `cat ~/.zshrc`
 - All files in `/etc/init.d/`
 - They are executed according to the run levels.³
 - Using systemd
 - `systemctl list-unit-files --type=service | grep enabled`
 - May not be used.

³ GeeksforGeeks (2018). Run Levels in Linux. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/run-levels-linux/> [Accessed 17 Oct. 2024].

LESSON 4 / DETECTING INTRUDERS IN YOUR SERVER

- Cron jobs: `cron` is a process that executes commands at a certain time and date

```
# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
```

- Usually abused to execute malware and other attacks when the attacker wants
- What is the meaning of those * chars?
 - <https://crontab.guru/>
- For the current user
 - Read and list cron entries
 - `crontab -l`
 - Edit your crontab
 - `crontab -e`
- For other users, check
 - `ll /var/spool/cron/crontabs/`
- For the system
 - Master cron of the system
 - `cat /etc/crontab`
 - You can have specials for generic system things
 - `vi /etc/cron.d/logcheck`
 - For specific moments without configuration
 - `/etc/cron.daily/`
 - `/etc/cron.hourly/`
 - `/etc/cron.monthly/`
 - `/etc/cron.weekly/`

Recap: To check if your computer has been attacked, you need to identify any

unauthorized modifications or additions. This is challenging and subject to change over time.

Password Time!!!

How to secure your computer? (15:19, 0m)

Securing a computer is a constant process and is extremely challenging. It requires monitoring of changes and periodic tests. Companies actually do a penetration test as part of the process to determine what to secure.

Hardening User access (15:19, 1m)

Let's harden the system so you have control over who logs in.

The best solution is to use SSH as the sole means of remote access.

VPNs are the second-best option, but with numerous providers, they can also pose security issues. Remote desktop programs are usually sufficient, but they can be too complex and may have vulnerabilities.

How to configure SSH to manage users? (15:20, 10m)

SSHD can be configured in `/etc/ssh/sshd_config`

- Configure to **forbid** the use of text passwords and force only public keys
- MOOC Students: Host Computer ▾ CTU Students: Host Computer ▾
 - If you don't have a pub/pri SSH key in your host computer, you can generate one with
 - `ssh-keygen`
 - You cannot put a password on your local private key if you trust your computer. If not, put one.
 - Copy your public key to the remote server.
 - MOOC Students: Host Computer ▾
 - `ssh-copy-id -p 2222 root@localhost`
 - CTU Students: Host Computer ▾

LESSON 4 / DETECTING INTRUDERS IN YOUR SERVER

- `ssh-copy-id -p <your-port> root@makalu.felk.cvut.cz`
- Try it. You should login now without putting your password
 - **MOOC Students: Host Computer** ▾
 - `ssh -p 2222 root@localhost`
 - **CTU Students: Host Computer** ▾
 - `ssh -p <your-port> root@makalu.felk.cvut.cz`
- Edit `/etc/ssh/sshd_config`, add
 - `PasswordAuthentication no`
-  ALWAYS CHECK IF IT WORKS **BEFORE** APPLYING!
-  ALWAYS HAVE A **BACKUP** CONNECTION WHILE YOU CHECK IN CASE IT FAILS!
 - Always create another user, set a password, and add it to the sudoers file (in the next classes). Then, try to log in **before** modifying your root account.
- How to choose which user can log in using SSH at all? Edit `/etc/ssh/sshd_config`
 - `AllowUsers <user to allow>`
 - `DenyUsers <user to deny>`
- Stop root from logging in to the system using SSH (be careful if you usually login as root, you will be out)
 - `PermitRootLogin no`
 - This affects both the password and the public key.
 - Do not do this in your CTU Dockers if you want to continue login with root user.
- You can also allow access to all the users in a group.
 - `AllowGroups sshusers`
- ALWAYS reload SSH for the modifications to take effect. You may be kicked out!
 - `/etc/init.d/ssh reload`

-  **Be careful!** Don't do `/etc/init.d/ssh restart`. This will restart SSH, and you will be **disconnected**; you will then need to log in again. The Docker is 'restarted'. But there is no data lost.
- One of the primary security concerns in remote terminals is the use of weak passwords and default passwords.
- We can solve this by implementing a check to ensure that new passwords are not weak. No user can be created with bad passwords.
- Edit this file and add
 - `vi /etc/pam.d/common-password`
 - `password requisite pam_pwquality.so retry=3 minlen=20 difok=3 dcredit=3 ucredit=3 lcredit=3`
 - `retry=3`: how many times the user gets to attempt again.
 - `minlen=20`: minimum length of the password
 - `difok=3`: checks the maximum number of reused characters compared to the user's old password.
 - `dcredit=3`: Minimum number of numerals
 - `ucredit=3`: Minimum number of uppercase characters
 - `lcredit=3`: Minimum number of lowercase characters.
 - Let's try it
 - `adduser test`
 - Ignore the errors; they are due to how Docker is run.
 - And try to put a simple password like 1234
 - `deluser test`

AI View:

Let's ask Clippy about the security of our config

- `cat /etc/ssh/sshd_config | bsy-clippy -u 'Check this configuration and only tell me if these settings are secure. Do not do commands' -r 3 -c -t 1`

LESSON 4 / DETECTING INTRUDERS IN YOUR SERVER

- -t 1: Temperature 1. 0 means more deterministic, 1 means more 'creative'
- -u: User prompt for this interaction (After system prompt from file)
- -r 3: Remember 3 chat interactions in the past of the chat.
- -c: Continue chatting after the stdin is read.
- After you are in interactive mode, you can ask to make them secure.

~~~~ ♡ First Break! ♡ ~~~~ (15:30, 10m)

### Hardening of files and directories (15:40, 12m)

CTU Students: Docker container. ▾ MOOC Students: SCL HackerLab ▾

Hardening refers to the process of tightening and securing permissions to prevent attackers from exploiting weak points.

In a weak system, attackers may be able to read, write, delete, or copy files that they should not.

Unfortunately, Linux does **not** come by default with a good file, directory, and user policy, so most Linux systems are badly configured.

- ⚠ If you put good permissions, are your files going to be protected **forever** and perfectly?
  - **Of course not.** The hardening of files and directories only works if the operating system is **controlling** the access.
  - If the hard disk is disconnected and read in another computer, then the permissions don't matter. Only encryption will help.
  - We will **not** delve into disk encryption, but we recommend checking **Veracrypt** and other options.
- Check the permissions in files

- `ls -alh /etc`

r: can read the file  
w: can write the file  
x: can execute the file

`-rwxrwxrwx 1 owneruser ownergroup 752B Sep 20 2018 file1`



Type:

- Regular file.
- b Block special file.
- c Character special file.
- d Directory.
- l Symbolic link.
- p FIFO.
- s Socket.

- Check the permissions in the directories

r: can list files  
w: can modify the structure of directory, like add or delete files  
x: can 'get in' with cd

`drwxrwxrwx 3 userowner groupowner 96B Apr 16 2016 directory1`



- For directory permission 'w,' you also need 'x' to make it work.
- There are some security permissions (`setuid`, `setgid`) that we are going to see in class 6. Not important for access restrictions to files.
- To change the permissions, use `chmod`

- `chmod [u|g|o][+|-][r|w|x] name`

- u: change the permissions of the user
- g: change the permissions of the group
- o: change the permissions of others
- +: add permissions
- -: delete permissions

## LESSON 4 / DETECTING INTRUDERS IN YOUR SERVER

- **r**: permission r
- **w**: permission w
- **x**: permission x
- **name**: name of file or directory
- **CTU Students: Docker container.** **MOOC Students: SCL HackerLab**

Let's try! Remember, you **are root** until the sudo command.

- **useradd luna**
- **mkdir /test**
- **chown root:luna /test**
  - Change the ownership.
  - The user owner of **/test** is **root**
  - The group owner of **/test** is **luna**
- **touch /test/file1**
  - As root, create a file
- **touch /test/file2**
  - As root, create a file
- **chown luna:luna /test/file2**
  - Make **luna** user and group own the second file
- **chmod g-w /test**
  - Probably not necessary, given that by default, the group owner has no write anyway.
- **ls -alh /test/**
  - Check permissions
  - Luna can not write in the **/test** folder
- **su - luna**
- **id**: See who you are
- **ls -alh /test/**
- **rm /test/file2**
  - **luna can not delete** the file that belongs to the **luna** user!

- This exemplifies the difference between being able to write in a directory or not.
- Stop being the `luna` user
  - `CTRL-D`

As a defense measure, you should check the permissions of:

- `/home`
- `/etc/shadow`
- `/etc/passwd`
- `/var/log`
- `/var/log/*`
- `/`
- `/tmp`
- `/home/*/.ssh/`

## Extra: Umask (15:52, 2m)

Umask is a system-global way to **force a default set of permissions for newly created files and directories**.

The mode mask contains the permission bits that should not be set on a newly created file. Hence, the mask acts as a filter to strip away default permission bits.

For **files**, the default permissions are **666**, and for **directories**, **777**.

- Check the current umask
  - `umask`
- Example to change it
  - `umask 026`
    - This means that when a new file is created, the final permissions will be 666 ‘minus’ 027, respecting the vertical position. That is:
      - $$\begin{array}{r} 666 \\ - 026 \\ \hline = 640 \end{array}$$
      - `-rW-r-----`
  - You can think of **6-0=6**, **6-2=4**, and **6-6=0**

- More easily, you can see umask in symbolic notation with
  - `umask -S`

### File Attributes (15:54, 5m)

CTU Students: Docker container. ▾ MOOC Students: SCL HackerLab ▾

Attributes are **metadata** stored in each file. It allows you to store **data** to modify what can be done or not done to the file.

Let's see some:

- `touch filetest`
- `lsattr filetest`

There are many attributes. The following are a few useful ones. (Not all filesystems support every attribute.)<sup>4</sup>

- **a** - Append-only: The file can only be opened for appending.
- **c** - compressed: Enable filesystem-level compression for the file.
- **i** - immutable: Cannot be modified, deleted, renamed, or linked to. It can only be set by the root user.
- **j** - Data journaling: Use the journal for both file data writes and metadata.
- **m** - no compression: Disable filesystem-level compression for the file.
- **A** - no atime update: The file's atime will not be modified.
- **C** - no copy on write: Disable copy-on-write for filesystems that support it.
- **e**: The file uses **extents** to map blocks on disk. Improve performance by allocating larger contiguous chunks.
- **s**: **secure deletion**
- **u**: **undeletable** (not in Docker with special Docker capabilities)
- You can change them with
  - `chattr +c filetest`
    - **+**: to add

---

<sup>4</sup> `lsattr(1)` - Linux manual page. (2025). Retrieved October 16, 2025, from Man7.org website: <https://man7.org/linux/man-pages/man1/lsattr.1.html>

- -: to delete
- c: the attribute to change. In this case, on-the-fly compression.
- The complete list is [here](#)<sup>5</sup>.
- We have a problem. In Docker, some of them are **restricted** and need further permissions when Docker is run.
  - If you run a Docker container, you need to run `docker with --cap-add CAP_LINUX_IMMUTABLE`. This applies to `+a` (append-only). For `+j` (data journaling), you require the `CAP_SYS_RESOURCE` capability.
- **Extra:** Let's try some more
  - `touch testfile`
  - `ls -alu --time-style=full-iso testfile`
    - `u`: show the time of last access and **not** the default time of the last modification.
  - `cat testfile`
    - i.e., access the file
  - `ls -alu --time-style=full-iso testfile`
  - `chattr +A testfile`
    - Put the attribute; do not update access time.
  - `lsattr testfile`
  - `cat testfile`
  - `ls -alu --time-style=full-iso testfile`
  - `chattr -A testfile`
    - Take it out
  - `lsattr testfile`
  - `cat testfile`
  - `ls -alu --time-style=full-iso testfile`



## Extended Attributes in files (15:59, 3m)

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

<sup>5</sup> chattr(1) - Linux manual page. (2025). Retrieved October 16, 2025, from Man7.org website: <https://man7.org/linux/man-pages/man1/chattr.1.html>

Apart from attributes associated with files, there are extended attributes that restrict how files and directories can be interacted with. Extended attributes are pairs of “**name:value**” associated with files and directories. They are just **values** that other processes use.

It's like adding some metadata value that is not restricted to just file system access.

There are four extended<sup>6</sup> *attribute spaces*: **security, system, trusted, and user**.

- To use the extended attributes, you need to install **attr** (already installed in our Docker)
- By default, there are no extended attributes
  - `touch file.txt`
  - `getfattr file.txt`
  - `getfattr -n user.test file.txt`
    - `-n` name. Dump the value of the named extended attribute.
  - `getfattr -d file.txt`
    - `-d`, Dump the values of all matched extended attributes.



### Use attributes in the User space (16:02, 3m)

**User**-extended attributes can be used to store arbitrary information about a file. It is up to you. Let's see:

- `touch file.txt`
- `setfattr -n user.note -v "this is mine" file.txt`
- `getfattr -n user.note file.txt`
- `getfattr -d -e base64 file.txt`
  - `-e` Encode values after retrieving them. Valid values of `en` are "text", "hex", and "base64".

The other attributes, **security**, **system**, and **trusted**, are used by the OS for other purposes, and you cannot modify them manually.

**Trusted** can be set and read only by the superuser, so a kind of secret message. Sorry, not in Docker as it is.

---

<sup>6</sup> to, C. (2005, December 18). Set of file system features. Retrieved October 16, 2025, from Wikipedia.org website: [https://en.wikipedia.org/wiki/Extended\\_file\\_attributes#Linux](https://en.wikipedia.org/wiki/Extended_file_attributes#Linux)

## Extra: Capabilities to processes (16:05, 15m)

Capabilities are a set of special permissions that the kernel allows to the **processes** created from executable **files**. Capabilities provide very fine-grained control over superuser and privileged permissions, **so you can avoid using root all the time**.

Traditionally, in UNIX systems, a process can be **privileged** (effective user ID 0) or an **unprivileged** process (eUID !=0).

- **Privileged** processes bypass all kernel permission checks.
- **Unprivileged** processes are fully checked in their permissions based on the process's credentials (usually effective UID, effective GID, and supplementary group list).
- After Linux 2.2, privileged processes can also be separated with capabilities.
- There are MANY capabilities<sup>7</sup>. For example:
  - **CAP\_NET\_ADMIN**. Perform various network-related operations
    - interface configuration
    - administration of IP firewall, masquerading, and accounting
    - Modify routing tables
    - bind to any address for transparent proxying
  - **CAP\_NET\_BIND\_SERVICE**
    - Bind a socket to Internet domain-privileged ports (port numbers less than 1024)
- Capabilities are implemented by using *extended attributes* in the *security space*!
- Example
  - `getcap /usr/bin/ping`
    - `/usr/bin/ping = cap_net_raw+ep`
    - Ping needs access to raw sockets to create the packets.
  - You can confirm it **is** a capability by trying to see the capabilities.

<sup>7</sup> Archlinux.org. (2022). capabilities(7) — Arch manual pages. [online] Available at: [https://man.archlinux.org/man/capabilities.7#File\\_capabilities](https://man.archlinux.org/man/capabilities.7#File_capabilities) [Accessed 17 Oct. 2024].

- `getfattr -n security.capability /usr/bin/ping`
  - `getfattr`: Removing leading '/' from absolute path names
  - `# file: usr/bin/ping`
  - `security.capability=0sAQAAAgAgAAAAAAAAAAAAAAAAAAAAA=`
- Let's try to give a file permission to use RAW sockets in the network
  - `touch testfile`
  - `setcap cap_net_raw+ep testfile`
  - `getcap testfile`
- Remove a capability
  - `setcap -r testfile`

### Access Control Lists (ACL) in files (16:20, 15m)

ACLs allow us to apply a more specific set of permissions to a file or directory without (necessarily) changing the traditional ownership and permission letters. **Solves the issue of multiple groups working on different files.** It is more granular.

- ACLs are implemented as *extended attributes* in the *system* space!
- You can list ACLs
  - `touch testfile`
  - `getfacl testfile`
    - `# file: testfile`
    - `# owner: root`
    - `# group: root`
    - `user::rw-`
    - `group::r--`
    - `other::r-`

**Situation:** We are super hackers, and we want a folder to be exclusive to root user. However, from time to time, we want an individual non-root user to contribute to it. But we **don't** want the user to be in the root group.

- Let's do it:
  - `mkdir /tmp/only-hackers`
  - `getfacl /tmp/only-hackers`
  - `adduser newbie`
    - Put a password
  - `tmux new -t newbie-session`
    - `su - newbie`
      - So we change users from root to newbie
    - `touch /tmp/only-hackers/mine`
      - *touch: cannot touch 'only-hackers/mine': No such file or directory*
    - `CTRL-b d` (get out of tmux)
- Let's make the newbie user able to write in the `only-hackers` folder
  - `setfacl -m newbie:rwX /tmp/only-hackers/`
    - `-m` is to modify
    - `newbie`: the username
    - `rwX`: what we want this user to be able to do
  - `getfacl /tmp/only-hackers/`
    - `# file: /tmp/only-hackers/`
    - `# owner: root`
    - `# group: root`
    - `user::rwX`
    - `user:newbie:rwX`
    - `group::r--`
    - `mask::rwX`
    - `other::---`
  - `tmux a -t newbie-session`
    - `touch /tmp/only-hackers/mine`
    - Should work!

## LESSON 4 / DETECTING INTRUDERS IN YOUR SERVER

Of course, we can always avoid an **evil** user from writing on it, even if it is a member of the group. You need a user called '**evil**'.

- `setfacl -m evil:- /tmp/only-hackers`

ACLs also allow you to force default users and groups for the new files created in a specific directory.

- If you create a file in `/tmp/only-hackers`, the default permissions are
  - `touch /tmp/only-hackers/test1`
  - `ll -alh /tmp/only-hackers/`
    - total 8.0K
    - drwxr-xr-x 2 root root 4.0K Oct 25 21:43 .
    - drwxrwxrwt 1 root root 4.0K Oct 25 21:43 ..
    - -rw-r--r-- 1 root root 0 Oct 25 21:43 test1

If we want all files in the `/tmp/only-hackers` directory to be set to `rw-----` (600) by default. But also, if they are folders, we want `rwx-----` (700). The group owner and others should not have any permissions. The command is:

- `setfacl -dRm u::rwX,g::- ,o::0 /tmp/only-hackers`
  - `-d`: be the default
  - `-R`: recursive
  - `-m`: modify
  - `[user/group/other]:[uid/gid]:[perms]`
  - `X`: capital X in the permissions means to apply the 'x' permission, but only if it is a directory or if the permission is already there. This is to prevent files from being executed automatically.
  - `0` (zero) is a permission in octal. The same is - as permissions.
- `touch /tmp/only-hackers/test1`
- `ll -alh /tmp/only-hackers/`
  - total 8.0K
  - drwxr-xr-x+ 2 root root 4.0K Oct 25 21:45 .
  - drwxrwxrwt 1 root root 4.0K Oct 25 21:43 ..

- `-rw-r--r-- 1 root root 0 Oct 25 21:43 test1`
- `-rw----- 1 root root 0 Oct 25 21:45 test2`
- Remember to delete the user `newbie`
  - `userdel newbie`

~~~~  **Second Break!**  ~~~~ (16:35, 10m)

How to delete unwanted users from our system? (16:45, 0m)

CTU Students: Docker container. ▾ MOOC Students: SCL HackerLab ▾

Imagine that you found a suspicious user in the system, and you want to delete all the files owned by that user. How to do it?

Delete files (16:45, 5m)

1. First, **back up** all their files in your system
 - a. To test: create a dummy user and file
 - i. `useradd -u 12000 luna`
 - ii. `touch /lunalunera`
 - iii. `chown luna:luna /lunalunera`
 - d. Find files belonging to the user you want to search
 - i. `find / -user luna`
 - e. Copy and tar the files automatically
 - i. `find / -user luna -exec tar -rvf /backup.tar {} \;`
 - f. Change the owner of the backup tar files
 - i. `chown root:root -R /backup.tar`
 - ii. You can keep this file or move it to another system for analysis.
 - g. Change permissions of tar file so only root can read it
 - i. `chmod o-rwx /backup.tar`
 - h. Alternity: Copy their files automatically for backup (you are the owner later)

- i. `mkdir /backup`
- ii. `find / -user luna -exec cp -r {} /backup \;`
- iii. Delete backup files in the folder
 1. `rm -rf /backup`

5. 🦴🔥 **Delete** their original files in your system from one user.

- i. `find / -user luna -exec rm -rf {} \;`

📁 Extra: Delete user accounts (16:50, 5m)

1. Delete their lines in `/etc/passwd` and `/etc/shadow`
 - a. Be careful with the editor's backup versions `/etc/shadow-` and `/etc/passwd-` if you edit them.
2. Delete their usernames in `/etc/group` (the system groups)
3. Remove their home folders (including their SSH keys)
 - b. `rm -rf /home/<change the user name>`
 - c. Check `/home/<user>/.ssh`
 - i. `/home/<user>/.ssh/authorized_keys`
6. You can also use the system command (will not delete the home folder)
 - a. `deluser luna`
 - b. `userdel luna`
7. You can use the system command AND delete the home folder:
 - a. `deluser --remove-home luna`

🛡️ Host-Based Intrusion Detection Systems: AIDE (16:55, 15m)

This is all good and pretty, but can't we just automate it? Yes, we can.

1. **Online** Students only, install AIDE. MOOC Students: SCL HackerLab ▾
 - a. `apt update`
 - b. `apt install aide`

- i. Select No configuration in mail setup
 - ii. Select to keep the local install version of the configuration
- 2. What is AIDE (Advanced Intrusion Detection Environment)?
 - a. It is a file integrity checker.
 - b. It creates a snapshot database of file and directory properties and then checks the system against that database.
 - i. For example: *file type, permissions, inode number, user, group, file size, mtime, ctime, atime, growing size, number of links, and link name.*

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

- 3. Let's try it
 - a. Your configuration in BSY Docker has been modified to only monitor the /etc directory, resulting in faster performance.
 - b. In StratoCyberLab is left by default because it is a smaller system.
 - c. To initialize the DB, run:
 - i. `aide --init --config=/etc/aide/aide.conf`
 - ii. A large system can take a considerable amount of time.... Like hours. So check the conf file and /etc/aide/aide.conf.d/ files.
 - d. Now, copy the **newly** generated DB with the following command to be the official DB:
 - i. `cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db`
- 4. Let's try to check modifications in your computer using the stored DB
 - a. `aide --check --config=/etc/aide/aide.conf`
- 5. Aide check only generates files in stdout
 - a. For example, if you still haven't gotten your output

```
Start timestamp: 2020-10-21 18:53:30 +0000 (AIDE 0.16.1)
AIDE found differences between database and filesystem!!
Verbose level: 6

Summary:
Total number of entries:41171
Added entries:          322
Removed entries:         1
Changed entries:         41
```

Added entries:

```
f+++++: /etc/cron.d/logcheck
d+++++: /etc/logcheck/cracking.d
```

Removed entries:

```
s-----: /run/screen/S-root/20447.aideinit
```

Changed entries:

```
f >.... . . . : /dev/tty10
f >.... mc..C.. .: /etc/aliases
d =.... mc.. . .: /etc/cron.d
d =.... mc.. . .: /etc/cron.daily
```

Detailed information about changes:

File: /etc/aliases

| | | |
|--------|-------------------------------------|----------------------------------|
| Size | : 185 | 200 |
| Mtime | : 2020-10-20 12:16:58 +0000 | 2020-10-21 18:40:37 +0000 |
| Ctime | : 2020-10-20 12:16:59 +0000 | 2020-10-21 18:40:37 +0000 |
| RMD160 | : Yc6MnN56GVq0H71ihHVwB19J9jE= | FXrOEAKQXccBS/U3q9dh5RB5omE= |
| TIGER | : aR6EXgbc5aUePggyMKc/4SzZti9LV+xB | onH7ExkjyZGDi9810iSjvxjYRVeF9aIk |
| SHA256 | : ebB3R8fA5ZglxGFkZYDqOnMJstQe6BJD | aglClQYou6prqNv5a/iRlDzUlwLlpOw |
| | Rl4xmo3p9ao= | OpVfqoN7BPY= |
| SHA512 | : qSE7OQJM10svrH3C4JqlGz5AxiomUYv00 | 35CJbU4S1IA7GuDcZ3UL17P568Yr50/G |
| | sQi5VTUOnm8JoXXm7ix57wM+uMciyKcR | 192QkRca9r54pIuT+ffgTP1+cXjS2IIg |
| | p3dRXE/10fh5AzcIAlnx2A== | 5tKtnv+dX9oJVQJWsX9yhw== |
| CRC32 | : M5knBg== | EV5Dsg== |
| HAVAL | : pJIfrZB7Asu4Gm8gYsVUI7bbwT/hvgu9 | RGNmunBTYDhCumsCi8Urx4RjHbE/mN4P |
| | HOjC1jUouuA= | GyDvyco4790= |
| GOST | : KitlDKOK7wz3yOCF4IXN88p40z1Y6knv | q7Iy8ZN3qYnZ0F4EQcY3GLY2YURdNI4R |
| | N2tuWT5WK8o= | MyWxhzkowEA= |

6. AIDE provides the **added, removed, and changed entries**.
7. An example line is “**f+++++**”. This string is formatted like “**YlZbpugamcinCAXS**”
 - a. **Y**: type of file. Can be: **f** for a regular file, **d** for a directory, **L** for a symbolic link, **D** for a character device, **B** for a block device, **F** for a FIFO, **s** for a Unix socket, and **?** otherwise
 - b. In ‘*Changed Entries*,’ there is also the change indicator

- i. The **Z** is replaced as follows: **A** = means that the size has not changed, a < reports a shrunk size and a > reports a grown size.
 - c. The other letters are the actual letters that will be output if the associated attribute has changed or:
 - i. "." for no change,
 - ii. "+" if the attribute has been added,
 - iii. "-" if it has been removed,
 - iv. ":" if the attribute is listed in ignore_list or
 - v. " " if the attribute has not been checked. (a newly created has all "+", removed file has all "-")
 - d. The attribute that is associated with each letter is as follows:
 - i. **l** → means that the link name has changed.
 - ii. **b** → means that the block count has changed.
 - iii. **p** → means that the permissions have changed.
 - iv. **u** → means that the uid has changed.
 - v. **g** → means that the gid has changed.
 - vi. **a** → means that the access time has changed.
 - vii. **m** → means that the modification time has changed.
 - viii. **c** → means that the change time has changed.
 - ix. **i** → means that the inode has changed.
 - x. **n** → means that the link count has changed.
 - xi. **C** → means that one or more checksums have changed.
 - xii. **A** → means that the access control list has changed.
 - xiii. **X** → means that the extended attributes have changed.
 - xiv. **S** → means that the SELinux attributes have changed.
8. If your check is ready, let's read it! Anything interesting? (17:30)
 9. If you want to update the DB with the changes (not now, will take time)

- a. `aide --update --config=/etc/aide/aide.conf`
- b. `cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db`

10. You can configure AIDE to run in CRON daily, and this generates the files in `/var/log/aide/`

11. Since any root user can modify the DB, you should copy it out.

Extra: Automated check of the security of a Linux (17:05, 10m)

CTU Students: Docker container. ▾ MOOC Students: SCL HackerLab ▾

There are tools that scan the entire system, checking for vulnerabilities and security issues. Unfortunately, there are not many options.

One of the latest is done by a company (as open source) and is called Lynis⁸

- `git clone https://github.com/CISOfy/lynis.git`
- `cd lynis`
- `./lynis audit system | less -RS`
- Let's see the output.

Extra: Automatic analysis of log files (17:15, 5m)

1. Logwatch
 - a. A program to analyze your system in general and logs in particular
 - b. `apt update`
 - c. `apt install logwatch`
 - d. `logwatch --service all --detail High --range All | less`
2. Of course, you can ask bsy-clippy
 - a. `logwatch --service all --detail High --range All| bsy-clippy -c -r 20 -u 'Analyze this output and tell me if there is any security concern' --profile cesnet`

⁸ CISOfy (2024). Lynis. [online] Cisofofy.com. Available at: <https://cisofofy.com/lynis/> [Accessed 17 Oct. 2024] Docker].

```

root@bsylabs:~$ logwatch --service all --detail High --range All | less | bsy-clippy -c -r 10 --profile cesnet -u 'anything security concern?'
bsy-clippy 0.3.0 | model 'qwen3-coder' via https://chat.ai.e-infra.cz/api (profile 'cesnet')
LLM (thinking): Yes, there are several security concerns:

1. **Repeated SSH brute-force attempts**: Numerous failed login attempts from multiple IPs, especially targeting root and soft accounts.
2. **Illegal user attempts**: Many attempts with usernames like admin, pepe, user, west from internal IPs.
3. **Weak password usage**: Multiple successful logins using password authentication (not keys).
4. **Deleted users still being targeted**: Indicates possible compromise or cleanup attempts.
5. **Sudo abuse attempts**: User "soft" repeatedly trying to run commands via sudo without permissions.
6. **SSH negotiation failures**: Old/insecure client connections being rejected, but still attempting.

Immediate actions:
- Disable password-based SSH logins, use keys only
- Block offending IPs via firewall
- Review and remove unnecessary user accounts
- Enforce strong passwords or disable weak accounts
- Monitor sudo access logs closely

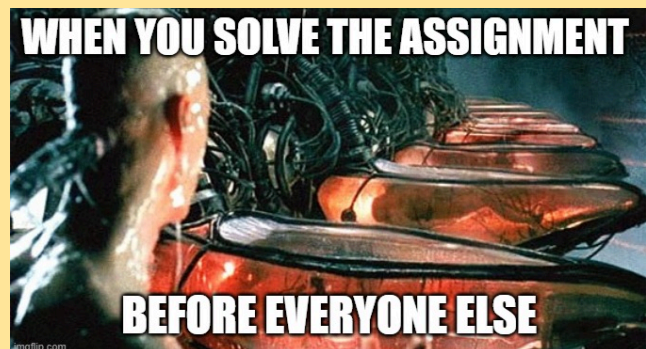
```

Assignment 5 (17:20, 5m)

There is a new Challenge in StratoCyberLab for everyone, called “Intrusion”.

CTU Students - Assignment 4 (5 Points)

1. This week's assignment will open on Thursday, 16th, 9 PM CEST.
2. The assignment has several parts. You will need to login to your Docker, investigate, and answer the questions in CTFd.
3. **Do not block user access by SSH (pass or public key).**
4. **Don't harden your dockers before the assignment starts.**



Class Feedback

By providing us with feedback after each class, you can help us make the next class even better!

<https://bit.ly/BSY-Feedback>

